

Change Delta

Every touched source file, old and new side by side — unchanged regions collapsed to four lines of context; deeply nested blocks are shown with their common leading indentation reduced (marked inline).

Old master @ ebe6f66

New feature/async-file-logging @ 6023033

4 files · +309 / -78 lines

2026-08-21

removed

added

changed within line

Sharp.Core/Bootstrap.cs		+19	-8
OLD · MASTER @ EBE6F66		NEW · FEATURE/ASYNC-FILE-LOGGING @ 6023033	
<pre>1 /* 2 * ModSharp 3 * Copyright (C) 2023-2026 Kxnrl. All Rights Reserved. 4 *</pre>		<pre>1 /* 2 * ModSharp 3 * Copyright (C) 2023-2026 Kxnrl. All Rights Reserved. 4 *</pre>	
... 64 unchanged lines ...			
<pre>69 { 70 ShutdownMonitor.Shutdown(); 71 StringPool.Shutdown(); 72 Console.WriteLine("MS: Shutdown Bootstrap!"); 73 } 74 #endregion 75 76</pre>		<pre>69 { 70 ShutdownMonitor.Shutdown(); 71 StringPool.Shutdown(); 72 Console.WriteLine("MS: Shutdown Bootstrap!"); 73 // Drain the async sink buffers before the process exits, 74 // otherwise the tail of Trace/Debug/Info/Warn logs is lost. 75 Log.CloseAndFlush(); 76 } 77 78 #endregion 79 80</pre>	
... 436 unchanged lines ...			
→ leading indentation reduced by 30 columns in this block (both sides)			
<pre>513 a.File(Path.Combine(logsDir, "Debug..log"), 514 rollingInterval: RollingInterval.Day, 515 outputTemplate: fileTemplate, 516 shared: true))) 517 518 .WriteTo.Logger(lc => lc.Filter 519 .ByIncludingOnly(e => e.Level is LogEventLevel.Information) 520 .WriteTo.File(Path.Combine(logsDir, "Info..log"), 521 rollingInterval: RollingInterval.Day, 522 outputTemplate: fileTemplate, 523 shared: false))) 524 525 .WriteTo.Logger(lc => lc 526 .Filter 527 .ByIncludingOnly(e => e.Level is LogEventLevel.Warning) 528 .WriteTo.File(Path.Combine(logsDir, "Warn..log"), 529 rollingInterval: RollingInterval.Day, 530 outputTemplate: fileTemplate, 531 shared: false))) 532 533 .WriteTo.Logger(lc => lc 534 .Filter 535 .ByIncludingOnly(e => e.Level >= LogEventLevel.Error) 536 .WriteTo.File(Path.Combine(logsDir, "Error..log"),</pre>		<pre>517 a.File(Path.Combine(logsDir, "Debug..log"), 518 rollingInterval: RollingInterval.Day, 519 outputTemplate: fileTemplate, 520 shared: true))) 521 // Info/Warn are written from the game thread in hot paths (spawn/kill/frame), 522 // so they use the async sink like Trace/Debug: a synchronous File sink 523 // performs a write+flush syscall per event on the calling thread. 524 .WriteTo.Logger(lc => lc.Filter 525 .ByIncludingOnly(e => e.Level is LogEventLevel.Information) 526 .WriteTo.Async(a => 527 a.File(Path.Combine(logsDir, "Info..log"), 528 rollingInterval: RollingInterval.Day, 529 outputTemplate: fileTemplate, 530 shared: false))) 531 .WriteTo.Logger(lc => lc 532 .Filter 533 .ByIncludingOnly(e => e.Level is LogEventLevel.Warning) 534 .WriteTo.Async(a => 535 a.File(Path.Combine(logsDir, "Warn..log"), 536 rollingInterval: RollingInterval.Day, 537 outputTemplate: fileTemplate, 538 shared: false))) 539 // Error stays synchronous on purpose: error events are rare (no frame-time 540 // pressure) and must reach the disk even if the process dies right after. 541 .WriteTo.Logger(lc => lc 542 .Filter 543 .ByIncludingOnly(e => e.Level >= LogEventLevel.Error) 544 .WriteTo.File(Path.Combine(logsDir, "Error..log"),</pre>	
... 127 unchanged lines ...			

```
661     return manager.InitModules();
662 }
663 }
664
```

```
672     return manager.InitModules();
673 }
674 }
675
```

Engine/src/logging.cpp

+284

-70

OLD · MASTER @ EBE6F66

NEW · FEATURE/ASYNC-FILE-LOGGING @ 6023033

```
1  /*
2   * ModSharp
3   * Copyright (C) 2023-2026 Kxnrl. All Rights Reserved.
4   *
5   * This file is part of ModSharp.
```

```
1  /*
2   * ModSharp
3   * Copyright (C) 2023-2026 Kxnrl. All Rights Reserved.
4   *
5   * This file is part of ModSharp.
```

... 20 unchanged lines ...

```
26 // sdk
27 #include "cstrike/interface/ICommandLine.h"
28
29 // crt
```

```
26 // sdk
27 #include "cstrike/interface/ICommandLine.h"
28
29 // crt
```

```
30 #include <cstdarg>
```

```
30 #include <atomic>
31 #include <chrono>
32 #include <condition_variable>
33 #include <cstdarg>
```

```
31 #include <cstring>
```

```
34 #include <cstdint>
35 #include <cstdio>
36 #include <cstring>
```

```
32 #include <filesystem>
```

```
37 #include <deque>
38 #include <filesystem>
```

```
33 #include <string>
```

```
39 #include <mutex>
40 #include <string>
41 #include <thread>
42 #include <unordered_map>
43 #include <utility>
```

```
34
35 #ifdef PLATFORM_WINDOWS
36 #    include <windows.h>
37 #endif
38
39 thread_local bool g_bInLoggingFlow;
40
41 std::string g_pLoggerMapName;
42
43 namespace fs = std::filesystem;
44
```

```
44
45 #ifdef PLATFORM_WINDOWS
46 #    include <windows.h>
47 #endif
48
49 thread_local bool g_bInLoggingFlow;
50
51 std::string g_pLoggerMapName;
52
53 namespace fs = std::filesystem;
54
```

```
55 constexpr const char* INFO_LOG_PATH = "../sharp/logs/info.log";
56 constexpr const char* ERROR_LOG_PATH = "../sharp/logs/error.log";
57 constexpr const char* FATAL_LOG_PATH = "../sharp/logs/fatal.log";
```

```
58 namespace {
```

```
60 struct AsyncLogEntry
```

```
61 {
62     std::string path;
63     std::string text;
64 };
```

```
65
66 // Log lines used to be written with fopen/fprintf/fflush/fclose on the calling
67 // thread, which is almost always the game thread. On slow or containerized
68 // storage a single flush costs multiple milliseconds and shows up as frame
69 // hitches. Lines are now fully formatted (including the timestamp) at the call
70 // site and handed to a dedicated writer thread that keeps the log files open.
71 class AsyncLogWriter
72 {
73     // When the queue is full new lines are dropped (and counted) instead of
```

```

75 // blocking the game thread on IO.
76 static constexpr size_t MAX_QUEUED_LINES = 4096;
77
78 public:
79 void Start()
80 {
81     std::lock_guard lock(m_Mutex);
82
83     if (m_bRunning)
84     {
85         return;
86     }
87
88     m_bRunning = true;
89     m_bStopRequested = false;
90     m_Thread = std::thread(&AsyncLogWriter::Run, this);
91 }
92
93 // Returns false if the writer is not running (early startup or after
94 // shutdown); the caller must then write synchronously. "text" is only
95 // consumed when true is returned.
96 bool TryEnqueue(const char* path, std::string&& text)
97 {
98     {
99         std::lock_guard lock(m_Mutex);
100
101         if (!m_bRunning || m_bStopRequested)
102         {
103             return false;
104         }
105
106         if (m_Queue.size() >= MAX_QUEUED_LINES)
107         {
108             m_nDroppedLines.fetch_add(1, std::memory_order_relaxed);
109             return true;
110         }
111
112         m_Queue.push_back({path, std::move(text)});
113     }
114
115     m_WakeUp.notify_one();
116
117     return true;
118 }
119
120 // Blocks until every queued line reached the disk or the timeout expired.
121
122 // Only used by the fatal path right before the process goes down.
123 void Flush(std::chrono::milliseconds timeout)
124 {
125     std::unique_lock lock(m_Mutex);
126
127     if (!m_bRunning)
128     {
129         return;
130     }
131
132     m_Drained.wait_for(lock, timeout, [this] { return m_Queue.empty() && !m_bWriting; });
133 }
134
135 // Must never be called while holding m_Mutex: join would deadlock against
136 // the writer thread taking the lock to grab the next batch.
137 void Shutdown()
138 {
139     {
140         std::lock_guard lock(m_Mutex);

```

```

140         std::lock_guard lock(m_Mutex);
141         if (!m_bRunning)
142         {
143             return;
144         }
145         m_bStopRequested = true;
146     }
147     m_Wakeup.notify_one();
148     m_Thread.join();
149     std::lock_guard lock(m_Mutex);
150     m_bRunning = false;
151 }
152
153 private:
154 void Run()
155 {
156     std::unordered_map<std::string, FILE*> files;
157
158     std::unique_lock lock(m_Mutex);
159
160     while (true)
161     {
162         m_Wakeup.wait(lock, [this] { return !m_Queue.empty() || m_bStopRequested; });
163
164         if (m_Queue.empty() && m_bStopRequested)
165         {
166             break;
167         }
168
169         std::deque<AsyncLogEntry> batch;
170         batch.swap(m_Queue);
171         m_bWriting = true;
172
173         // Write without holding the lock so producers never wait on IO.
174         lock.unlock();
175         WriteBatch(files, batch);
176         lock.lock();
177
178         m_bWriting = false;
179
180         if (m_Queue.empty())
181         {
182             m_Drained.notify_all();
183         }
184     }
185
186     lock.unlock();
187
188     for (const auto& [path, file] : files)
189     {
190         if (file != nullptr)
191         {
192             fclose(file);
193         }
194     }
195 }
196
197 void WriteBatch(std::unordered_map<std::string, FILE*>& files, std::deque<AsyncLogEntry>& batch)
198 {
199     if (const auto dropped = m_nDroppedLines.exchange(0, std::memory_order_relaxed); dropped > 0)
200     {
201         char notice[128];
202         snprintf(notice, sizeof(notice), "[AsyncLogWriter] dropped %llu log line(s): queue was full\n",
203 
```

```

205     auto& entry = static_cast<AsynLogEntry>(&dropped);
206     static_cast<unsigned long long>(dropped));
207     batch.push_front({ERROR_LOG_PATH, notice});
208 }
209 for (const auto& entry : batch)
210 {
211     auto& file = files[entry.path];
212     if (file == nullptr)
213     {
214         file = fopen(entry.path.c_str(), "a+");
215         if (file == nullptr)
216         {
217             continue;
218         }
219     }
220     fwrite(entry.text.data(), sizeof(char), entry.text.size(), file);
221 }
222 // One flush per touched file per batch instead of one per line.
223 for (const auto& [path, file] : files)
224 {
225     if (file != nullptr)
226     {
227         fflush(file);
228     }
229 }
230
231 std::mutex          m_Mutex;
232 std::condition_variable m_WakeUp;
233 std::condition_variable m_Drained;
234 std::deque<AsynLogEntry> m_Queue;
235 std::thread         m_Thread;
236 std::atomic<uint64_t> m_nDroppedLines{0};
237 bool                m_bRunning      = false;
238 bool                m_bStopRequested = false;
239 bool                m_bWriting      = false;
240 };
241
242 AsyncLogWriter g_AsyncLogWriter;
243
244 void WriteLogLineSync(const char* path, const std::string& text)
245 {
246     const auto logFile = fopen(path, "a+");
247     if (logFile != nullptr)
248     {
249         fwrite(text.data(), sizeof(char), text.size(), logFile);
250         fflush(logFile);
251         fclose(logFile);
252     }
253 }
254
255 void WriteLogLine(const char* path, std::string&& text)
256 {
257     if (!g_AsyncLogWriter.TryEnqueue(path, std::move(text)))
258     {
259         WriteLogLineSync(path, text);
260     }
261 }
262
263 // The map name is read on the calling thread on purpose: g_pLoggerMapName is
264 // mutated on map change and must not be touched from the writer thread.
265 std::string BuildLogLine(const char* timestamp, const char* level, const char* function, const char* text)

```

```

45 void CreateLogging()
46 {
47     const fs::path path = ".././sharp/logs";
48     if (!fs::exists(path))
49     {
50         fs::create_directory(path);
51     }
52 }
53
54 void WriteTextToFile(const char* path, const char* text)
55 {

```

```

128     g_bInLoggingFlow = true;
129     ConColorMsg({233, 0, 0, 255}, "L<Engine> [%s] | Fatal Error %s\n%s\n\n", timestamp, g_pLoggerMapName.c_str(), text);
130     g_bInLoggingFlow = false;
131
132     // 写入日志文件
133     const auto logFile = fopen(".././sharp/logs/fatal.log", "a+");
134     if (logFile != nullptr)
135     {

```

```

271 {
272     char header[512];
273
274     if (function != nullptr)
275     {
276         if (!g_pLoggerMapName.empty())
277         {
278             snprintf(header, sizeof(header), "[%s] | %s | %s\n", timestamp, level, function, g_pLoggerMapName.c_str());
279         }
280         else
281         {
282             snprintf(header, sizeof(header), "[%s] | %s | %s\n", timestamp, level, function);
283         }
284     }
285     else
286     {
287         if (!g_pLoggerMapName.empty())
288         {
289             snprintf(header, sizeof(header), "[%s] | %s\n", timestamp, level, g_pLoggerMapName.c_str());
290         }
291         else
292         {
293             snprintf(header, sizeof(header), "[%s] | %s\n", timestamp, level);
294         }
295     }
296
297     std::string line;
298     line.reserve(strlen(header) + strlen(text) + 2);
299     line.append(header);
300     line.append(text);
301     line.append("\n\n");
302
303     return line;
304 }
305 } // namespace
306
307 void CreateLogging()
308 {
309     const fs::path path = ".././sharp/logs";
310     if (!fs::exists(path))
311     {
312         fs::create_directory(path);
313     }
314 }

```

```

315     g_AsyncLogWriter.Start();
316 }
317
318 void ShutdownLogging()
319 {
320     g_AsyncLogWriter.Shutdown();
321 }
322
323 void WriteTextToFile(const char* path, const char* text)
324 {
325

```

... 72 unchanged lines ...

```

398     g_bInLoggingFlow = true;
399     ConColorMsg({233, 0, 0, 255}, "L<Engine> [%s] | Fatal Error %s\n%s\n\n", timestamp, g_pLoggerMapName.c_str(), text);
400     g_bInLoggingFlow = false;
401
402     // The process dies below: give queued lines a chance to reach the disk,
403     // then write the fatal record itself synchronously.
404     g_AsyncLogWriter.Flush(std::chrono::milliseconds(1000));
405

```

136	<code>fprintf(logFile, "[%s] Fatal Error %s\n", timestamp, g_pLoggerMapName.c_str());</code>	406	<code>WriteLogLineSync(FATAL_LOG_PATH, BuildLogLine(timestamp, "Fatal Error", nullptr, text));</code>
137	<code>fprintf(logFile, "%s\n\n", text);</code>		
138	<code>fflush(logFile);</code>		
139	<code>fclose(logFile);</code>		
140	<code>}</code>		
141		407	
142	<code>#ifdef PLATFORM_WINDOWS</code>	408	<code>#ifdef PLATFORM_WINDOWS</code>
143	<code>if (IsDebuggerPresent())</code>	409	<code>if (IsDebuggerPresent())</code>
144	<code>{</code>	410	<code>{</code>
... 35 unchanged lines ...			
180	<code>g_bInLoggingFlow = true;</code>	446	<code>g_bInLoggingFlow = true;</code>
181	<code>ConColorMsg({233, 0, 0, 255}, "L<Engine> [%s] Fatal Error %s\n%s\n\n", timestamp, g_pLoggerMapName.c_str(), text);</code>	447	<code>ConColorMsg({233, 0, 0, 255}, "L<Engine> [%s] Fatal Error %s\n%s\n\n", timestamp, g_pLoggerMapName.c_str(), text);</code>
182	<code>g_bInLoggingFlow = false;</code>	448	<code>g_bInLoggingFlow = false;</code>
183		449	
184	<code>const auto logFile = fopen("../sharp/logs/fatal.log", "a+");</code>	450	<code>// Fatal records always go to the disk synchronously: the process usually</code>
185	<code>if (logFile != nullptr)</code>	451	<code>// does not live long enough for a queued write.</code>
186	<code>{</code>	452	<code>WriteLogLineSync(FATAL_LOG_PATH, BuildLogLine(timestamp, "Fatal Error", nullptr, text));</code>
187	<code>if (!g_pLoggerMapName.empty())</code>		
188	<code>fprintf(logFile, "[%s] Fatal Error %s\n", timestamp, g_pLoggerMapName.c_str());</code>		
189	<code>else</code>		
190	<code>fprintf(logFile, "[%s] Fatal Error\n", timestamp);</code>		
191			
192	<code>fprintf(logFile, "%s\n\n", text);</code>		
193	<code>fflush(logFile);</code>		
194	<code>fclose(logFile);</code>		
195	<code>}</code>		
196	<code>}</code>	453	<code>}</code>
197		454	
198	<code>void LogError(const char* message, ...)</code>	455	<code>void LogError(const char* message, ...)</code>
199	<code>{</code>	456	<code>{</code>
... 10 unchanged lines ...			
210	<code>g_bInLoggingFlow = true;</code>	467	<code>g_bInLoggingFlow = true;</code>
211	<code>ConColorMsg({233, 0, 0, 255}, "L<Engine> [%s] Error %s\n%s\n\n", timestamp, g_pLoggerMapName.c_str(), text);</code>	468	<code>ConColorMsg({233, 0, 0, 255}, "L<Engine> [%s] Error %s\n%s\n\n", timestamp, g_pLoggerMapName.c_str(), text);</code>
212	<code>g_bInLoggingFlow = false;</code>	469	<code>g_bInLoggingFlow = false;</code>
213		470	
214	<code>const auto logFile = fopen("../sharp/logs/error.log", "a+");</code>	471	<code>WriteLogLine(ERROR_LOG_PATH, BuildLogLine(timestamp, "Error", nullptr, text));</code>
215	<code>if (logFile != nullptr)</code>		
216	<code>{</code>		
217	<code>if (!g_pLoggerMapName.empty())</code>		
218	<code>fprintf(logFile, "[%s] Error %s\n", timestamp, g_pLoggerMapName.c_str());</code>		
219	<code>else</code>		
220	<code>fprintf(logFile, "[%s] Error\n", timestamp);</code>		
221			
222	<code>fprintf(logFile, "%s\n\n", text);</code>		
223	<code>fflush(logFile);</code>		
224	<code>fclose(logFile);</code>		
225	<code>}</code>		
226	<code>}</code>	472	<code>}</code>
227		473	
228	<code>void LogInfo(const char* message, ...)</code>	474	<code>void LogInfo(const char* message, ...)</code>
229	<code>{</code>	475	<code>{</code>
... 10 unchanged lines ...			
240	<code>g_bInLoggingFlow = true;</code>	486	<code>g_bInLoggingFlow = true;</code>
241	<code>ConColorMsg(Color(0, 255, 255, 255), "L<Engine> [%s] Information %s\n%s\n\n", timestamp, g_pLoggerMapName.c_str(), text);</code>	487	<code>ConColorMsg(Color(0, 255, 255, 255), "L<Engine> [%s] Information %s\n%s\n\n", timestamp, g_pLoggerMapName.c_str(), text);</code>
242	<code>g_bInLoggingFlow = false;</code>	488	<code>g_bInLoggingFlow = false;</code>
243		489	
244	<code>const auto logFile = fopen("../sharp/logs/info.log", "a+");</code>	490	<code>WriteLogLine(INFO_LOG_PATH, BuildLogLine(timestamp, "Information", nullptr, text));</code>
245	<code>if (logFile != nullptr)</code>		
246	<code>{</code>		
247	<code>if (!g_pLoggerMapName.empty())</code>		
248	<code>fprintf(logFile, "[%s] Information %s\n", timestamp, g_pLoggerMapName.c_str());</code>		
249	<code>else</code>		
250	<code>fprintf(logFile, "[%s] Information\n", timestamp);</code>		
251	<code>}</code>		

252	fprintf(logFile, "%s\n\n", text);		
253	fflush(logFile);		
254	fclose(logFile);		
255	}		
256	}	491	}
257		492	
258	void LogFuncError(const char* function, const char* message, ...)	493	void LogFuncError(const char* function, const char* message, ...)
259	{	494	{
... 10 unchanged lines ...			
270	g_bInLoggingFlow = true;	505	g_bInLoggingFlow = true;
271	ConColorMsg({233, 0, 0, 255}, "L<Engine> [%s] Error %s %s\n%s\n\n", timestamp, function, g_pLoggerMapName.c_str(), text);	506	ConColorMsg({233, 0, 0, 255}, "L<Engine> [%s] Error %s %s\n%s\n\n", timestamp, function, g_pLoggerMapName.c_str(), text);
272	g_bInLoggingFlow = false;	507	g_bInLoggingFlow = false;
273		508	
274	const auto logFile = fopen(".././sharp/logs/error.log", "a+");	509	WriteLogLine(ERROR_LOG_PATH, BuildLogLine(timestamp, "Error", function, text));
275	if (logFile != nullptr)		
276	{		
277	if (!g_pLoggerMapName.empty())		
278	fprintf(logFile, "[%s] Error %s %s\n", timestamp, function, g_pLoggerMapName.c_str());		
279	else		
280	fprintf(logFile, "[%s] Error %s\n", timestamp, function);		
281			
282	fprintf(logFile, "%s\n\n", text);		
283	fflush(logFile);		
284	fclose(logFile);		
285	}		
286	}	510	}
287		511	
288	void LogFuncInfo(const char* function, const char* message, ...)	512	void LogFuncInfo(const char* function, const char* message, ...)
289	{	513	{
... 10 unchanged lines ...			
300	g_bInLoggingFlow = true;	524	g_bInLoggingFlow = true;
301	ConColorMsg(Color(0, 255, 255, 255), "L<Engine> [%s] Information %s %s\n%s\n\n", timestamp, function, g_pLoggerMapName.c_str(), text);	525	ConColorMsg(Color(0, 255, 255, 255), "L<Engine> [%s] Information %s %s\n%s\n\n", timestamp, function, g_pLoggerMapName.c_str(), text);
302	g_bInLoggingFlow = false;	526	g_bInLoggingFlow = false;
303		527	
304	const auto logFile = fopen(".././sharp/logs/info.log", "a+");	528	WriteLogLine(INFO_LOG_PATH, BuildLogLine(timestamp, "Information", function, text));
305	if (logFile != nullptr)	529	}
306	{		
307	if (!g_pLoggerMapName.empty())		
308	fprintf(logFile, "[%s] Information %s %s\n", timestamp, function, g_pLoggerMapName.c_str());		
309	else		
310	fprintf(logFile, "[%s] Information %s\n", timestamp, function);		
311		530	
312	fprintf(logFile, "%s\n\n", text);		
313	fflush(logFile);		
314	fclose(logFile);		
315	}		
316	}		

Engine/src/Logging.h		+4	-0
OLD · MASTER @ EBE6F66		NEW · FEATURE/ASYNC-FILE-LOGGING @ 6023033	
1	/*	1	/*
2	* ModSharp	2	* ModSharp
3	* Copyright (C) 2023-2026 Kxnrl. All Rights Reserved.	3	* Copyright (C) 2023-2026 Kxnrl. All Rights Reserved.
4	*	4	*
... 113 unchanged lines ...			
118	void FatalError(const char* message, ...) MS_FMTFUNCTION(1, 2);	118	void FatalError(const char* message, ...) MS_FMTFUNCTION(1, 2);
119		119	

120	void CreateLogging();	120	void CreateLogging();
121		121	
		122	// Drains the queue, joins the writer thread and closes the log files.
		123	// File logging falls back to synchronous writes afterwards.
		124	void ShutdownLogging();
		125	
122	void WriteTextToFile(const char* path, const char* text);	126	void WriteTextToFile(const char* path, const char* text);
123		127	
124	#define LOG(...) ConsoleMessage(__FUNCTION__, __VA_ARGS__)	128	#define LOG(...) ConsoleMessage(__FUNCTION__, __VA_ARGS__)
125	#define WARN(...) ConsoleWarning(__FUNCTION__, __VA_ARGS__)	129	#define WARN(...) ConsoleWarning(__FUNCTION__, __VA_ARGS__)
126	#define FLOG(...) LogFuncInfo(__FUNCTION__, __VA_ARGS__)	130	#define FLOG(...) LogFuncInfo(__FUNCTION__, __VA_ARGS__)
127	#define FERROR(...) LogFuncError(__FUNCTION__, __VA_ARGS__)	131	#define FERROR(...) LogFuncError(__FUNCTION__, __VA_ARGS__)
128		132	
129	#endif	133	#endif

Engine/src/modsharp.cpp				+2	-0
OLD · MASTER @ EBE6F66		NEW · FEATURE/ASYNC-FILE-LOGGING @ 6023033			
1	/*	1	/*		
2	* ModSharp	2	* ModSharp		
3	* Copyright (C) 2023-2026 Kxnrl. All Rights Reserved.	3	* Copyright (C) 2023-2026 Kxnrl. All Rights Reserved.		
4	*	4	*		
... 265 unchanged lines ...					
270	{	270	{		
271	coreclr::Shutdown();	271	coreclr::Shutdown();		
272		272			
273	HookManager::Uninstall();	273	HookManager::Uninstall();		
		274	ShutdownLogging();		
		275			
274	}	276	}		

New file in this pull request (no previous version to compare)

DISCLAIMER.txt	Warranty and liability disclaimer with operational notes. Per-commit details live in the pull request description; this document is hosted outside the PR branch.
--------------------------------	---